

Code Explanation

GA Prompt Optimiser for LLM Code Generation

Evolving System Prompts on Codeforces Problems

Group 1

Computational Intelligence — Spring 2026

April 2026

Deep-dive walkthrough of every file, every function, and every call chain.

Includes exact function signatures, cross-file data-flow diagrams, and step-by-step worked examples tracing real data through the pipeline.

Backend: Google Cloud Vertex AI (Gemini 2.5 Flash & Pro).

Designed for readers with no prior knowledge of genetic algorithms or LLM APIs.

Contents

1	Introduction	2
1.1	What This Project Does	2
1.2	The Codebase at a Glance	2
1.3	Module Dependency Graph	2
1.4	How to Read This Document	4
2	Architecture & Data Flow	5
2.1	The Two-Model Architecture	5
2.2	Master Execution Pipeline	5
2.3	Data Types Flowing Between Modules	7
2.4	The API Call Budget	8
2.5	Fitness Score: What Is Being Optimised?	8
3	Configuration: <code>config.py</code>	9
3.1	Purpose	9
3.2	File at a Glance	9
3.3	Full Source Code	9
3.4	Block-by-Block Explanation	10
3.4.1	Path Resolution	10
3.4.2	Dataset Constants	11
3.4.3	Genetic Algorithm Constants	11
3.4.4	Vertex AI Constants	12
3.4.5	Convergence Detection	12
4	Seed Prompts: <code>seeds.py</code>	14
4.1	Purpose	14
4.2	File at a Glance	14
4.3	The 15 Strategies at a Glance	14
4.4	The Safety Assert	14
4.4.1	How Seeds Become Individuals	14
5	Dataset Loading: <code>dataset.py</code>	16
5.1	Purpose	16
5.2	File at a Glance	16
5.3	Problem Distribution	16
5.4	Function-by-Function Analysis	16
5.4.1	<code>load_problems()</code> $\rightarrow$ <code>list[dict]</code>	16
5.4.2	<code>split(problems, evolution_size, holdout_size, seed=42)</code>	17
5.4.3	<code>reserved_train_problems(problems, seed=42)</code> $\rightarrow$ <code>list[dict]</code>	18
6	Code Evaluation: <code>evaluator.py</code>	19
6.1	Purpose	19
6.2	File at a Glance	19
6.3	Function-by-Function Analysis	19
6.3.1	<code>_kill_group(proc)</code>	19
6.3.2	<code>extract_code(response: str) $\rightarrow$ str</code>	20

6.3.3	<code>_compile(code: str) → (bin_path None, error_msg)</code>	20
6.3.4	<code>_run_one(bin_path, stdin_str, timeout) → Optional[str]</code>	21
6.3.5	<code>run_tests_verbose(code, inputs, outputs, timeout=2) → dict</code>	21
7	Cost Tracking: <code>cost_tracker.py</code>	23
7.1	Purpose	23
7.2	The PRICING Table	23
7.3	<code>CallRecord</code> — One Call’s Data	23
7.4	<code>CostTracker</code> — The Streaming Singleton	24
8	LLM Calls: <code>llm.py</code>	25
8.1	Purpose	25
8.2	File at a Glance	25
8.3	Output Directives	25
8.4	<code>_get_api_client()</code>	26
8.5	<code>_call_realtime_api()</code> — Core API Wrapper	27
8.6	<code>generate_code(system_prompt, question) → str</code>	28
8.7	<code>crossover()</code> and the Three Mutation Operators	28
9	Genetic Algorithm: <code>ga.py</code>	30
9.1	Purpose	30
9.2	File at a Glance	30
9.3	<code>Individual</code> — The Chromosome Dataclass	30
9.4	<code>_eval_code_for_problem()</code> and <code>_eval_one_problem()</code>	31
9.5	<code>compute_fitness(gen, individual, problems) → float</code>	32
9.6	<code>_assign_failure_examples(individual, collected_failures)</code>	32
9.7	RNG State Serialization	32
9.8	<code>tournament_select</code> , <code>roulette_select</code> , and <code>breed</code>	33
9.9	<code>run()</code> — The Main GA Loop	34
10	Orchestration: <code>main.py</code>	37
10.1	Purpose	37
10.2	File at a Glance	37
10.3	Function-by-Function Analysis	37
10.3.1	<code>save_checkpoint(run_dir, gen, population, stats, rng_state=None)</code>	37
10.3.2	<code>_find_latest_run()</code> and <code>_load_resume_state()</code>	38
10.3.3	<code>evaluate_on_holdout(best_prompt, holdout, run_dir) → float</code>	38
10.3.4	<code>main()</code> — The 8-Step Pipeline	38
10.4	Output Files	40
11	Genealogy: <code>genealogy.py</code>	41
11.1	Purpose	41
11.2	File at a Glance	41
11.3	Algorithm	41
	Glossary	43

Chapter 1

Introduction

1.1 What This Project Does

This project uses a **Genetic Algorithm (GA)** to automatically discover the best “system prompt” for a Large Language Model (LLM) that generates **C++ code** for competitive programming problems.

A **system prompt** is a paragraph of plain-English instructions prepended to every LLM call, shaping how it thinks before it sees the coding problem. Examples: “*Think like a competitive programmer*” or “*Always check edge cases before writing code.*”

The GA treats each system prompt as a **chromosome** — a candidate solution that can be scored (fitness evaluated), combined with another (crossover), and lightly altered (mutation), exactly as in biological evolution. Over 12 generations the GA evolves a population of 5 prompts, keeping the best-performing ones and combining their strategies, until it converges on a prompt that makes the LLM solve the most Codeforces problems correctly.

The Core Insight

Manual prompt engineering is trial-and-error. A human tries one prompt, tests it, tweaks it, and tries again. This project automates that loop: the GA *searches* the entire space of possible English instruction paragraphs by treating test-case pass rates as the fitness signal. The LLM’s own creativity (via crossover and mutation calls) is used to *generate* new prompts, while its code-generation ability is what is being *optimised*.

Key design choice: C++ and Vertex AI

The target model is asked to produce a **complete C++ program** (`int main()`, reads from `stdin`, writes to `stdout`). Evaluation compiles the code with `g++ -O2 -std=c++23` and runs the resulting binary. All LLM calls go through the **Google Cloud Vertex AI** REST API via the `google-genai` Python SDK, authenticated with a service account JSON file — no subscription or interactive login is required.

1.2 The Codebase at a Glance

The project consists of 9 Python files:

1.3 Module Dependency Graph

Listing 1.1: Module import graph (arrows = imports).

```
main.py
|-- imports --> config, dataset, evaluator, ga, llm, genealogy
|-- imports --> cost_tracker (singleton: tracker)
|-- imports --> seeds (SEED_PROMPTS list)

ga.py
```

File	Lines	Public items	Role
config.py	83	25 constants	Single source of truth for all settings
seeds.py	137	1 list	The 15 generation-zero C++ chromosomes
dataset.py	253	3 functions	Load & stratified-split 305 Codeforces problems
evaluator.py	295	4 functions	Extract C++ code; compile; run binary
cost_tracker.py	244	2 classes	Stream token counts; compute cost; write JSON
llm.py	470	7 functions	All Vertex AI Gemini calls via google-genai SDK
ga.py	717	10 items	The full evolutionary loop (parallel eval, resume, selection schemes)
main.py	545	6 functions	Orchestrate, checkpoint, resume, save results
genealogy.py	154	1 function	Ancestry report for the best evolved prompt

Table 1.1: Project file summary.

```

|-- imports --> config, evaluator, llm

llm.py
|-- imports --> config
|-- imports --> cost_tracker (singleton: tracker, PRICING dict)
|-- uses      --> google.genai SDK (Vertex AI realtime API)
|-- uses      --> google.oauth2.service_account (auth)

evaluator.py
|-- imports --> os, re, signal, subprocess, tempfile, time (stdlib only)
|-- imports --> config

dataset.py
|-- imports --> json, random, sys, pathlib, collections (stdlib only)
|-- imports --> config

cost_tracker.py
|-- imports --> json, threading, time, dataclasses, datetime, pathlib (stdlib
)

genealogy.py
|-- imports --> json, pathlib (stdlib only)

config.py
|-- imports --> pathlib (stdlib only)

seeds.py
|-- (no imports)

```

Key Import Insight

`evaluator.py`, `dataset.py`, `cost_tracker.py`, `genealogy.py`, `config.py`, and `seeds.py` have *zero external dependencies*. `llm.py` is the *only* file that touches the network, and it does so by calling the `google-genai` Python SDK which connects to Google Cloud Vertex AI via HTTPS, authenticated with a service account JSON credential file.

1.4 How to Read This Document

Each chapter covers one file and follows this structure:

1. **Purpose** — plain-English summary of what the file does and why it exists.
2. **File at a Glance** — table of every function/class with one-line descriptions.
3. **Full Source Code** — the actual source with line numbers.
4. **Function-by-Function Analysis** — for every function:
 - Exact signature (all parameters, types, defaults)
 - What calls it and what it calls (the call chain)
 - Step-by-step walkthrough of the body
 - Return value with a concrete example
 - Edge cases and error handling
5. **Worked Example** — real data traced through the function.

Colour coding for boxes: **blue** = concept/theory; **orange** = warning/gotcha; **green** = function specification; **yellow** = worked example.

Chapter 2

Architecture & Data Flow

2.1 The Two-Model Architecture

A critical design choice is that *two separate Gemini models* are used for two completely different purposes:

Role	Model	Temperature	Used for
Target (the subject)	gemini-2.5-flash	0.0	Generating C++ code (fitness eval)
Optimizer (the tool)	gemini-2.5-pro	0.7	Crossover & mutation of prompts

Table 2.1: The two-model design. Temperature is enforced via the API `GenerateContentConfig`.

Why Two Models?

Target (gemini-2.5-flash): We are searching for the prompt that makes *this specific model* write the best C++ code. The goal is to fix the model and optimise the prompt. Temperature 0.0 gives deterministic, reproducible code output.

Optimizer (gemini-2.5-pro): Crossover and mutation require *creativity* — merging two prompts into a coherent paragraph, inventing a new instructional strategy. The Pro model is more capable for creative reasoning tasks. Temperature 0.7 adds controlled randomness to generate diverse offspring.

Thinking tokens

Both models are configured with a `ThinkingConfig`: `thinking_budget=512` for fitness calls and `thinking_budget=2048` for operator calls. Gemini 2.5 Flash/Pro can use “thinking tokens” (internal chain-of-thought) before generating the final output. These are billed at the output token rate and tracked separately in `usage_metadata.thoughts_token_count`.

2.2 Master Execution Pipeline

Below is the complete call tree for a single run.

Listing 2.1: Complete call tree for one GA run.

```
main.main()
|
|-- [pre-flight: Vertex AI credentials]
|   check config.SERVICE_ACCOUNT_PATH exists
|   import google.genai, google.oauth2.service_account (verify installed)
|   read service-account.json -> verify project_id, client_email
|
|-- [pre-flight: dataset dirs]
```

```

|     config.APPS_TRAIN.exists() and config.APPS_TEST.exists()
|
|-- dataset.load_problems()
|     Reads 305 pre-selected Codeforces problems from _SELECTED dict
|     Returns: list[dict] (id, question, inputs, outputs, rating)
|
|-- dataset.split(all_problems, 65, 200, seed=42)
|     Stratified: harder train (>=1800) always in evolution
|     Regular tiers 800-1700 drawn per _EVOLUTION_PER_TIER counts
|     Returns: (evolution[65], holdout[200], rest[~40])
|
|-- tracker.set_log_path(run_dir/"calls.jsonl")
|-- tracker.set_live_path(run_dir/"costs_live.json")
|-- tracker.set_costs_live_path(Costs/"cost_<run>.json")
|-- llm.set_error_log_path(run_dir/"llm_errors.log")
|
+-- ga.run(SEED_PROMPTS[:5], evolution_pool, run_dir, on_generation=...,
          selection_scheme=args.scheme, elitism_count=1 if scheme=="elitism"
          else None)
|
| [GEN 0: evaluate all 5 seeds, sequential]
| for seed_idx, ind in enumerate(population):
|     [restore from seed checkpoint if it exists]
|     ind.fitness = compute_fitness(gen=0, ind, problems)
|     ThreadPoolExecutor(max_workers=5):
|         _eval_one_problem(gen, ind, prob, idx, total) x65
|         time.sleep(random.uniform(0, 1)) -- stagger
|         llm.generate_code(ind.prompt, prob["question"])
|         client.models.generate_content(
|             model="gemini-2.5-flash",
|             contents=user_msg,
|             config=GenerateContentConfig(
|                 system_instruction=system,
|                 max_output_tokens=8192,
|                 temperature=0.0,
|                 thinking_config=ThinkingConfig(budget=512)))
|         tracker.record(phase,model,in,out,cost)
|         returns text (raw LLM output)
|         evaluator.extract_code(response) -> C++ string
|         evaluator.run_tests_verbose(code, inputs, outputs, 2)
|         _compile(code) -> g++ -O2 -std=c++23 -> binary
|         for each test: _run_one(bin, stdin, 2) -> stdout
|         _outputs_match(actual, expected) -> bool
|         returns {score, reason, compile, test_cases, ...}
|         _log_evaluation(entry) -> evaluations_genNN.jsonl
|         returns (score, failure_example | None)
|         _assign_failure_examples(ind, collected_failures)
|         save seed checkpoint: seed_NN_<uid>.json
|
| [GENS 1..12]
|     population.sort(by fitness, descending)
|     stats = {generation, best, mean, worst, std, cv, best_uid,
|             best_prompt}
|     history.append(stats)
|     on_generation(gen-1, population, stats, rng.getstate())
|     -> save_checkpoint(run_dir, gen-1, pop, stats, rng_state)
|         writes: results/run_*/generation_NN.json (includes rng_state)
|
| [convergence check: CV < 0.01 for 3 consecutive gens -> early stop]
|
| new_pop = [] -- ELITISM_COUNT=0, no frozen elites
| offspring_idx = 0

```



```

|     while len(new_pop) < 5:  -- breed 5 offspring
|         p1 = tournament_select(population, rng)  -- best of 3
|         p2 = tournament_select(population, rng)
|         allow_mut = (offspring_idx >= CROSSOVER_ONLY_SLOTS=2)
|         child = breed(p1, p2, rng, allow_mutation=allow_mut)
|         [80%] llm.crossover(a.prompt, b.prompt) -> merged prompt
|         [if allow_mut:
|             [25%] llm.mutate_inject(prompt, failure_examples)
|             [15%] llm.mutate_delete(prompt)
|             [10%] llm.mutate_rephrase(prompt)]
|         returns Individual(prompt, parent_a, parent_b, operators,
|                             intermediate_steps)
|         new_pop.append(child); offspring_idx += 1
|
|     for ind in new_pop:  -- evaluate all 5 offspring
|         ind.fitness = compute_fitness(gen, ind, problems)
|     population = new_pop
|
| +-- returns (best_Individual, history[list of dicts])
|
|-- main.evaluate_on_holdout(best.prompt, holdout[200])
|     ThreadPoolExecutor(max_workers=5): 200 calls, same pipeline
|     Returns: float (holdout fitness)
|
|-- genealogy.generate_report(run_dir, best.uid)
|     reads generation_NN.json files -> builds ancestry chain
|     writes genealogy_report.txt + genealogy_tree.json
|
|-- (run_dir/"summary.json").write_text(...)
+-- tracker.flush(extra={...})
|     writes: Code/Costs/cost_<run>.json
|     Returns: Path

```

2.3 Data Types Flowing Between Modules

Producer	Consumer	Type	Description
seeds.py	ga.py	list[str]	15 initial prompt strings
dataset.py	ga.py, main.py	list[dict]	Problem dicts (id, question, inputs, outputs, rating)
llm.py	ga.py, main.py	str	Raw LLM response text
evaluator.py	ga.py, main.py	str	Extracted C++ code string
evaluator.py	ga.py, main.py	dict	Verbose test result (score, compile, test cases)
ga.py	main.py	Individual	Best chromosome (prompt + fitness + metadata)
ga.py	main.py	list[dict]	History: best/mean/-worst/std/cv per generation
llm.py	cost_tracker.py	int, float	token counts + cost_usd per API call
cost_tracker.py	main.py	Path	Path to written cost JSON file
main.py	genealogy.py	Path, str	run_dir + best uid

Table 2.2: Data flowing between modules.

2.4 The API Call Budget

One full run (Pop=5, Gen=12, Evo=65, Holdout=200) makes the following API calls:

Call type	Count	Model	Formula
Fitness (gen 0 seeds)	325	Flash	5 seeds $\times$ 65 problems
Fitness (gens 1–12)	3,900	Flash	12 gens $\times$ 5 offspring $\times$ 65 problems
Holdout evaluation	200	Flash	1 best prompt $\times$ 200 problems
Crossover	≤ 48	Pro	12 gens $\times$ 5 offspring $\times$ 80%
Mutate inject	≤ 9	Pro	12 gens $\times$ 3 mut-eligible $\times$ 25%
Mutate delete	≤ 5	Pro	12 gens $\times$ 3 mut-eligible $\times$ 15%
Mutate rephrase	≤ 4	Pro	12 gens $\times$ 3 mut-eligible $\times$ 10%
Total Flash	$\approx 4,425$	—	—
Total Pro	≈ 66	—	—

Table 2.3: Approximate API call counts per full run. “Mut-eligible” = offspring_idx $\geq$ CROSOVER_ONLY_SLOTS=2.

CROSOVER_ONLY_SLOTS

The first CROSOVER_ONLY_SLOTS=2 offspring bred each generation receive crossover only — no mutation operators fire on them. This preserves the best genetic material more faithfully in the early slots while still allowing recombination. The remaining 3 offspring per generation can receive any combination of inject, delete, and rephrase mutations in addition to crossover.

2.5 Fitness Score: What Is Being Optimised?

The fitness of one prompt = the average partial-credit score across 65 Codeforces problems. The partial credit for one problem = (test cases passed) / (total test cases).

$$fitness(p) = \frac{1}{65} \sum_{i=1}^{65} \frac{testspassedonproblem_i using prompt p}{totaltestsonproblem_i}$$

A fitness of 1.0 means the prompt caused the LLM to write C++ code that passed *every* test case on *every* problem. Infrastructure failures (all retries exhausted) score -1.0 and are included in the mean, creating a small penalty for unreliable prompts.

Chapter 3

Configuration: config.py

3.1 Purpose

Every hyperparameter and file path lives in this one file. No other file hard-codes any numbers or paths. Authentication is done via a service account JSON file (`service-account.json`) that grants access to Google Cloud Vertex AI — no interactive login or API key string is required.

3.2 File at a Glance

3.3 Full Source Code

Listing 3.1: `config.py` — complete source.

```
1  """
2  config.py -- All hyperparameters and paths in one place.
3  Edit here; everything else reads from this module.
4  """
5
6  from pathlib import Path
7
8  # -----
9  # Paths
10 # -----
11 ROOT          = Path(__file__).parent
12 APPS_TRAIN    = ROOT / "Dataset" / "APPS" / "train"
13 APPS_TEST     = ROOT / "Dataset" / "APPS" / "test"
14 RESULTS_DIR   = Path(__file__).parent / "results"
15
16 # -----
17 # Dataset
18 # -----
19 # 305 pre-selected Codeforces problems (stdin/stdout, full C++
20 #   programs).
21 # Evolution pool: 65 stratified (60 from tiers 800-1700 + 5 harder
22 #   1800/1900).
23 EVOLUTION_SIZE = 65      # fixed pool for fitness evaluation during GA
24 HOLDOUT_SIZE   = 200     # held-out for final within-distribution test
25 RANDOM_SEED    = 42
26 EARLY_STOP_FAILURES = 3  # stop running test cases after this many
27                       # consecutive failures
28
29 # -----
30 # Genetic Algorithm
31 # -----
32 POPULATION_SIZE = 5
33 GENERATIONS     = 12
```

```

31 ELITISM_COUNT          = 0      # no frozen elites -- all slots bred via
    crossover+mutation
32 CROSSOVER_ONLY_SLOTS = 2      # first N offspring get crossover only (no
    mutation)
33 TOURNAMENT_SIZE        = 3      # contestants per tournament selection draw
34
35 CROSSOVER_RATE          = 0.8
36
37 # Independent per-individual mutation probabilities (can all apply at
    once)
38 MUT_INJECT_PROB        = 0.25
39 MUT_DELETE_PROB         = 0.15
40 MUT_REPHRASE_PROB       = 0.10
41
42 # -----
43 # Google Cloud Vertex AI -- Gemini backend
44 # -----
45 SERVICE_ACCOUNT_PATH    = Path(__file__).parent / "service-account.json"
46 GCP_REGION               = "us-central1"
47
48 # Model being optimised -- generates code, evaluated for fitness.
49 TARGET_MODEL             = "gemini-2.5-flash"
50
51 # Model used for crossover / mutation meta-prompts.
52 OPTIMIZER_MODEL          = "gemini-2.5-pro"
53
54 TARGET_TEMPERATURE        = 0.0
55 OPTIMIZER_TEMPERATURE    = 0.7
56
57 API_FALLBACK_MAX_TOKENS_CODE = 8192    # max output tokens for code
    generation
58 API_FALLBACK_MAX_TOKENS_OPS  = 700     # crossover / mutation outputs
    are short
59
60 # -----
61 # Convergence detection (CV = std / mean, per generation)
62 # -----
63 CONVERGENCE_CV_THRESHOLD = 0.01        # 1% variation -> converged
64 CONVERGENCE_PATIENCE     = 3           # consecutive stagnant generations
    to confirm
65 CONVERGENCE_MIN_MEAN     = 0.01        # only check CV when mean fitness
    exceeds this
66
67 # -----
68 # Code execution (g++ compilation + binary runs)
69 # -----
70 EXEC_TIMEOUT             = 2           # seconds per test-case binary run
71 COMPILE_TIMEOUT          = 15          # seconds allowed for g++ compilation
72 PARALLEL_EVALS           = 5           # problems evaluated simultaneously per
    individual

```

3.4 Block-by-Block Explanation

3.4.1 Path Resolution

```

ROOT      = Path(__file__).parent
            -- __file__ = ".../Code/config.py"
            -- .parent  = ".../Code/"
            -- ROOT     = ".../Code/"

APPS_TRAIN = ROOT / "Dataset" / "APPS" / "train"
            = ".../Code/Dataset/APPS/train"
APPS_TEST  = ROOT / "Dataset" / "APPS" / "test"
            = ".../Code/Dataset/APPS/test"
RESULTS_DIR = Path(__file__).parent / "results"
            = ".../Code/results"

SERVICE_ACCOUNT_PATH = Path(__file__).parent / "service-account.json"
            = ".../Code/service-account.json"

```

Both APPS_TRAIN and APPS_TEST are verified by `main.main()` before the GA starts. SERVICE_ACCOUNT_PATH is checked for existence and parsed for `project_id` as the very first pre-flight step.

3.4.2 Dataset Constants

- EVOLUTION_SIZE = 65 — The GA evaluates every prompt on exactly these 65 problems. Keeping the set fixed across all generations is critical: if the problems changed, a prompt might score higher simply because it got easier problems. The actual set is determined by stratified sampling (`dataset.split()`), not a simple first-65 slice.
- HOLDOUT_SIZE = 200 — After the GA finishes, the *best* prompt is tested on these 200 *never-seen* problems to measure generalisation.
- RANDOM_SEED = 42 — Passed to `dataset.split()` and `ga.run()`'s `random.Random` instance to make the experiment fully reproducible.
- EARLY_STOP_FAILURES = 3 — In `evaluator.run_tests_verbose()`: after 3 consecutive test failures, the test loop exits early to save time.

3.4.3 Genetic Algorithm Constants

No elitism (ELITISM_COUNT = 0)

Unlike classic GA designs, the current version does not freeze any individuals across generations. Every generation, all 5 slots are filled by new offspring from crossover and mutation. Elitism was removed because with a small population of 5, freezing 2 elites leaves only 3 slots for exploration — too restrictive. The best individual is still preserved implicitly because tournament selection strongly favours it as a parent.

CROSSOVER_ONLY_SLOTS = 2

The first 2 offspring bred each generation get crossover but no mutation operators. This creates two “conservative” children that faithfully combine parent strategies. The remaining 3 offspring also receive the three independent mutation operators, which introduce novel strategies. This hybrid balances exploitation (crossover-only) and exploration (mutation).

Tournament size: exploration vs. exploitation

TOURNAMENT_SIZE = 3 means we draw 3 random contestants and select the best. With only 5 individuals in the population, a tournament of 3 draws a large fraction of the pool, creating strong selection pressure toward the current best.

3.4.4 Vertex AI Constants

`SERVICE_ACCOUNT_PATH` points to a Google Cloud service account JSON file that grants access to the Vertex AI API. The file is read once at startup by `llm._get_api_client()`; credentials are then held in a singleton `google.genai.Client` object shared across all threads.

`GCP_REGION = "us-central1"` is the Vertex AI region. Both `gemini-2.5-flash` and `gemini-2.5-pro` are confirmed available in this region.

Unlike the previous Claude CLI backend, temperature is now *enforced* via `GenerateContentConfig(temperature=...)` in every API call. `TARGET_TEMPERATURE = 0.0` gives deterministic, reproducible code generation; `OPTIMIZER_TEMPERATURE = 0.7` adds creative variation to crossover and mutation.

3.4.5 Convergence Detection

The GA can stop early if the population becomes statistically homogeneous:

- `CONVERGENCE_CV_THRESHOLD = 0.01`: CV (coefficient of variation = std/mean) below 1% means all individuals are performing nearly identically.
- `CONVERGENCE_PATIENCE = 3`: the CV must stay below the threshold for 3 *consecutive* generations before early stopping triggers.
- `CONVERGENCE_MIN_MEAN = 0.01`: CV is only meaningful when mean fitness is positive. If all individuals are failing ($\text{mean} \approx 0$), the early-stop check is skipped to avoid false convergence signals.

Name	Type	Description
ROOT	Path	Project root (one level above <code>Code/</code>)
APPS_TRAIN	Path	Path to APPS/Codeforces training data
APPS_TEST	Path	Path to APPS/Codeforces test data
RESULTS_DIR	Path	Where run output folders are created
SERVICE_ACCOUNT_PATH	Path	Path to service-account.json credential file
GCP_REGION	str	Vertex AI region (" <code>us-central1</code> ")
TARGET_MODEL	str	Gemini model for code generation (<code>gemini-2.5-flash</code>)
OPTIMIZER_MODEL	str	Gemini model for crossover/mutation (<code>gemini-2.5-pro</code>)
TARGET_TEMPERATURE	float	Temperature for code generation (0.0, enforced)
OPTIMIZER_TEMPERATURE	float	Temperature for operators (0.7, enforced)
API_FALLBACK_MAX_TOKENS_CODE	int	Max output tokens for code gen (8192)
API_FALLBACK_MAX_TOKENS_OPS	int	Max output tokens for operators (700)
EVOLUTION_SIZE	int	Problems used for fitness during GA (65)
HOLDOUT_SIZE	int	Problems reserved for final test (200)
RANDOM_SEED	int	Seed for all random decisions (42)
EARLY_STOP_FAILURES	int	Stop after N consecutive test failures (3)
POPULATION_SIZE	int	Chromosomes alive at any time (5)
GENERATIONS	int	Breeding cycles (12)
ELITISM_COUNT	int	Top individuals copied unchanged (0)
CROSSOVER_ONLY_SLOTS	int	First N offspring get crossover only (2)
TOURNAMENT_SIZE	int	Contestants per tournament selection (3)
CROSSOVER_RATE	float	Probability crossover fires per offspring (0.8)
MUT_INJECT_PROB	float	Probability inject mutation fires (0.25)
MUT_DELETE_PROB	float	Probability delete mutation fires (0.15)
MUT_REPHRASE_PROB	float	Probability rephrase mutation fires (0.10)
CONVERGENCE_CV_THRESHOLD	float	CV < this $\Rightarrow$ stagnant (0.01)
CONVERGENCE_PATIENCE	int	Stagnant generations before early stop (3)
CONVERGENCE_MIN_MEAN	float	Don't check CV if mean fitness below this (0.01)
EXEC_TIMEOUT	int	Seconds per test-case binary run (2)
COMPILE_TIMEOUT	int	Seconds allowed for g++ compilation (15)
PARALLEL_EVALS	int	Problems evaluated simultaneously (5)

Table 3.1: `config.py`: all exported names.

Chapter 4

Seed Prompts: `seeds.py`

4.1 Purpose

The GA must start with a population. `seeds.py` provides 15 hand-crafted instruction prompts — the “generation zero” chromosomes. Each emphasises a different cognitive strategy for solving C++ competitive programming problems, ensuring the GA starts with genuinely diverse material to crossover and mutate. The pool of 15 seeds is larger than the current `POPULATION_SIZE = 5`; `main.py` uses only `SEED_PROMPTS[:POPULATION_SIZE]`, so the first 5 seeds form the initial population.

Why diversity in seeds matters

If all 5 seeds said the same thing, crossover would merely re-shuffle identical text. By providing 15 different strategies — edge-case thinking, algorithm identification, pseudocode-first, constraint-awareness, etc. — crossover can combine genuinely different approaches from the start, providing the GA with rich exploratory material.

4.2 File at a Glance

Name	Type	Description
<code>SEED_PROMPTS</code>	<code>list[str]</code>	15 diverse C++ instruction strings (gen-0 chromosomes)

Table 4.1: `seeds.py`: exported names.

4.3 The 15 Strategies at a Glance

4.4 The Safety Assert

```
assert len(SEED_PROMPTS) >= 5, \
    f"Need at least 5 seed prompts; got {len(SEED_PROMPTS)}"
```

If you accidentally remove seeds below the minimum, Python immediately raises `AssertionError` when `seeds.py` is imported — before any API calls are made. The threshold is 5 (= `POPULATION_SIZE`) rather than exactly 15, to allow the seed pool to be extended without changing the assert.

4.4.1 How Seeds Become Individuals

In `ga.run()` [`ga.py`]:

```
seeds = SEED_PROMPTS[:config.POPULATION_SIZE]  # first 5 of the 15 seeds
population = [Individual(prompt=p, operators="seed") for p in seeds]
# Each seed string becomes an Individual with:
```


#	Strategy	Core instruction focus
1	Structured analysis	Analyse input/output format, constraints, edge cases before coding
2	Terse and direct	Full C++ solution; g++ -std=c++23; no commentary
3	Competitive programmer	Find algorithm pattern (greedy, DP, graph, binary search, math)
4	Step-by-step reasoning	4-step: understand, identify, boundary conditions, implement
5	Example-driven	Trace sample inputs manually before writing code
6	Conversational	Full C++ source with main; compile and handle all edge cases
7	Edge-case defender	n=0, n=1, all equal, max values, integer overflow
8	Algorithm-first	Determine algorithm + complexity before writing code
9	Specification-heavy	5-point spec: cin, cout, g++ -O2 -std=c++23, all cases, exact format
10	Pseudocode-then-code	English sketch first, then translate to C++
11	Constraint-aware	If n can be 10^5 , $O(n^2)$ will TLE
12	Defensive / robust	Adversarial inputs at extreme constraint boundaries
13	Readability-first	Meaningful variable names; correct beats clever
14	Verify-against-examples	Mental verification against sample inputs before finalising
15	Direct and focused	Entire C++ program; reads stdin; writes stdout; passes all tests

Table 4.2: The 15 seed strategies. Prompts 1–5 form the initial population (POPULATION_SIZE=5).

```
# prompt = the seed string
# fitness = -1.0 (unevaluated sentinel)
# uid = random 8-char hex (e.g. "a3f7c2b1")
# parent_a = "" (empty: seeds have no parents)
# parent_b = ""
# operators = "seed"
# intermediate_steps = {}
# failure_examples = []
```

Chapter 5

Dataset Loading: `dataset.py`

5.1 Purpose

This file loads 305 pre-selected Codeforces competitive programming problems from disk and splits them into three non-overlapping groups: the **evolution pool** (65 problems for fitness evaluation during the GA), the **held-out set** (200 problems for the final generalisation test), and the **rest** (remaining ≈ 40 problems).

The split is **stratified**: harder problems (rating ≥ 1800) are always included in the evolution pool; regular tiers 800–1700 are drawn proportionally per `_EVOLUTION_PER_TIER` counts. This is fundamentally different from the old simple shuffle.

5.2 File at a Glance

Item	Type	Description
<code>_SELECTED</code>	dict	Hard-coded problem IDs by split then rating
<code>_EVOLUTION_PER_TIER</code>	dict	How many problems to draw from each tier (800–1700)
<code>_RESERVED_PER_TIER</code>	dict	How many leftover problems go into the reserved subfolder
<code>load_problems()</code>	function	Load all 305 problems from disk; returns list of dicts
<code>split(...)</code>	function	Stratified split into evolution (65) / holdout (200) / rest
<code>reserved_train_problems()</code>	function	Return 35 “reserved” train problems for the reserved set

Table 5.1: `dataset.py`: all items.

5.3 Problem Distribution

5.4 Function-by-Function Analysis

5.4.1 `load_problems()` $\rightarrow$ `list[dict]`

`load_problems()` $\rightarrow$ `list[dict]`

Called by: `main.main()`

Calls: `json.loads()`, `Path.read_text()`, `Path.exists()`

Returns: `list[dict]` — one dict per successfully loaded problem (≤ 305)

Tier	Rating	Train	Test	Total	Evo draw
Tier 1	800	29	21	50	7
Tier 1	1000	10	40	50	6
Tier 2	1100	14	19	33	6
Tier 2	1200	23	10	33	7
Tier 2	1300	23	11	34	7
Tier 3	1400	27	5	32	7
Tier 3	1500	33	0	33	7
Tier 4	1600	17	0	17	6
Tier 4	1700	18	0	18	7
Tier 5	1800	3	0	3	3 (always)
Tier 5	1900	2	0	2	2 (always)
Total		199	106	305	65

Table 5.2: Problem distribution. Harder problems (1800/1900) are always in the evolution pool.

Iterates over the hard-coded `_SELECTED` dict (split $\rightarrow$ rating $\rightarrow$ folder IDs). For each folder it reads `question.txt` and `input_output.json`. Skips problems with missing files, bad JSON, or empty test cases.

Worked Example — A real problem dict

```
{
  "id":      "train/0002",
  "question": "Given n integers on a line, compute their sum.\n"
              "Input: first line is n; second line has n integers.\n"
              "Output: a single integer.\nExample input:\n3\n1 2 3\n"
              "Example output:\n6",
  "inputs":  ["3\n1 2 3\n", "1\n0\n", "5\n-1 -2 3 4 5\n"],
  "outputs": ["6\n",          "0\n",    "9\n"],
  "rating":   800,
}
# inputs[i] = raw string piped to the compiled C++ binary's stdin
# outputs[i] = expected string on stdout (whitespace-normalised at compare
              time)
```

5.4.2 split(problems, evolution_size, holdout_size, seed=42)

```
split(...) -> (evolution[65], holdout[200], rest[ 40])
```

Called by: `main.main()`

Parameters:

problems — full list from `load_problems()` (≤ 305 dicts)

evolution_size — accepted for API compat but actual size is stratified

holdout_size — size of the held-out test set (200)

seed — RNG seed (42)

Returns: tuple[list, list, list] — (evolution, holdout, rest)

The split is **stratified**, not a simple shuffle:

```
# Step 1: separate harder train problems (1800/1900) -- always in evolution
harder_train = [p for p in problems
```

```

        if p["id"].startswith("train/") and p["rating"] >= 1800]

# Step 2: group regular train problems by rating tier
regular_by_tier = defaultdict(list)
for p in problems:
    if p["id"].startswith("train/") and p["rating"] < 1800:
        regular_by_tier[p["rating"]].append(p)

# Step 3: draw per-tier using _EVOLUTION_PER_TIER counts (seed=42 shuffle)
rng = random.Random(seed)
evo_regular = []
for rating in sorted(regular_by_tier):
    tier_probs = regular_by_tier[rating].copy()
    rng.shuffle(tier_probs)
    n = _EVOLUTION_PER_TIER.get(rating, 0)
    evo_regular.extend(tier_probs[:n])
    # deterministic
    # 6 or 7 depending on tier

evolution = harder_train + evo_regular
# 5 + 60 = 65 problems

# Step 4: everything else goes to holdout / rest
evo_ids = {p["id"] for p in evolution}
remaining = [p for p in problems if p["id"] not in evo_ids]
rng.shuffle(remaining)
holdout = remaining[:holdout_size]
rest = remaining[holdout_size:]
# 200 problems
# ~40 problems

```

Stratification vs. simple shuffle

The old design shuffled all 300 problems uniformly and took the first 100. This meant harder problems (1700+ rating) might or might not appear in the evolution pool depending on the random seed — making the fitness signal unstable across experiments. The new stratified design guarantees:

- All 5 harder problems (1800/1900) are *always* in the evolution pool.
- Each regular tier contributes a predictable count (6 or 7 problems).
- The fitness signal has a controlled, consistent difficulty distribution.

5.4.3 reserved_train_problems(problems, seed=42) → list[dict]

reserved_train_problems(problems, seed=42) → list[dict]

Called by: setup_reserved.py (not the main GA loop)

Returns: 35 train problems (800–1700) for the reserved subfolder

Returns the problems that were *not* selected for evolution but were still curated: the 35 problems (from tiers 800–1700) drawn using `_RESERVED_PER_TIER` counts, taken immediately after the evolution picks in the same tier shuffle order. Together with the 65 active evolution problems this gives 100 curated train problems total; the remaining 205 are from the test split.

Chapter 6

Code Evaluation: `evaluator.py`

6.1 Purpose

When the LLM generates a response, we need to:

1. **Extract** the C++ code from the LLM's raw text response.
2. **Compile** the extracted code with `g++ -O2 -std=c++23`.
3. **Execute** the compiled binary against each test case, piping the input string to stdin and comparing stdout against the expected output.
4. **Score** the solution as (tests passed) / (total tests) with early stopping.

This file has no LLM calls and no external dependencies beyond the Python standard library. Process-group killing (`_kill_group`) ensures that orphaned child processes from timeouts never block the evaluator.

6.2 File at a Glance

Item	Type	Description
<code>_MAX_IO_LEN</code>	constant	Max chars per test case in JSONL (500)
<code>_kill_group(proc)</code>	private	Kill entire process group of <code>proc</code> (prevents orphans)
<code>extract_code(response)</code>	function	Pull C++ code out of raw LLM text
<code>_normalise(text)</code>	private	Strip trailing whitespace per line
<code>_outputs_match(a, e)</code>	private	Compare normalised actual vs. expected
<code>_compile(code)</code>	private	Write to temp <code>.cpp</code> ; run <code>g++ -std=c++23</code> ; return binary path
<code>_run_one(bin, stdin, t)</code>	private	Run compiled binary with one input string
<code>run_tests_verbose(...)</code>	function	Full pipeline: compile + run + early-stop + detailed dict
<code>run_tests(...)</code>	function	Thin wrapper returning only the float score

Table 6.1: `evaluator.py`: all items.

6.3 Function-by-Function Analysis

6.3.1 `_kill_group(proc)`

```
_kill_group(proc: subprocess.Popen) -> None
```

Called by: `_compile()`, `_run_one()` on timeout

Sends SIGKILL to the entire process group of `proc`:

```
def _kill_group(proc):
```

```

try:
    os.killpg(os.getpgid(proc.pid), signal.SIGKILL)
except (ProcessLookupError, PermissionError):
    try:
        proc.kill() # fallback: kill only the direct process
    except Exception:
        pass

```

Without this, a timed-out C++ program that spawned child processes could leave them running and writing to the pipe, causing the parent's `communicate()` to hang. Both `_compile()` and `_run_one()` use `preexec_fn=os.setsid` when launching subprocesses, which places them in a new session (= their own process group), making `os.killpg` safe to call.

6.3.2 `extract_code(response: str) → str`

Tries three strategies in priority order:

```

# Strategy 1: tagged C++ fence
m = re.search(r"``(?:cpp|c|c\+\+)\s*\n([\s\S]*)``", response, re.IGNORECASE)
if m: return m.group(1).strip()

# Strategy 2: any fence
m = re.search(r"``\s*\n?([\s\S]*)``", response)
if m: return m.group(1).strip()

# Strategy 3: raw text (assume entire reply is code)
return response.strip()

```

Because both Gemini models are configured with a `_CODE_GEN_DIRECTIVE` that explicitly forbids markdown fences, the LLM typically returns raw C++ code (strategy 3). Strategies 1 and 2 handle the cases where the model ignores the directive.

6.3.3 `_compile(code: str) → (bin_path | None, error_msg)`

```
_compile(code) -> (str | None, str)
```

```
Calls: tempfile.mkstemp(), subprocess.Popen(["g++", "-O2", "-std=c++23", ...])
```

```

_compile(code):
    src_fd, src_path = tempfile.mkstemp(suffix=".cpp")
    bin_path = src_path[:-4] -- strip .cpp extension

    write code to src_path via os.fdopen(src_fd)

    proc = subprocess.Popen(
        ["g++", "-O2", "-std=c++23", "-o", bin_path, src_path],
        stdout=PIPE, stderr=PIPE, text=True,
        preexec_fn=os.setsid) -- new session -> own process group

    try:
        _, stderr = proc.communicate(timeout=COMPILE_TIMEOUT) -- 15s limit
    except subprocess.TimeoutExpired:
        _kill_group(proc); proc.communicate()
        return None, "compilation timed out"

    if proc.returncode != 0:
        return None, stderr[:500] -- compile error text

    return bin_path, "" -- success: binary exists

```

```

finally:
    os.unlink(src_path)                -- always delete .cpp
    if not _success: os.unlink(bin_path) -- delete binary only on failure

```

g++ -std=c++23

The compiler standard was upgraded from c++17 to c++23 to support modern Codeforces solutions that use C++23 features (`std::print`, `std::ranges`, etc.). The seed prompts also reference c++23 in several places (seeds 2 and 9). g++ 13+ (available on Ubuntu 24.04) supports `-std=c++23`.

6.3.4 `_run_one(bin_path, stdin_str, timeout) → Optional[str]`

```

_run_one(bin_path, stdin_str, timeout):
    proc = subprocess.Popen(
        [bin_path],
        stdin=PIPE, stdout=PIPE, stderr=DEVNULL,
        text=True,
        preexec_fn=os.setsid)          -- own process group for clean kill

    try:
        stdout, _ = proc.communicate(input=stdin_str, timeout=timeout)
        return stdout
    except subprocess.TimeoutExpired:
        _kill_group(proc); proc.communicate()
        return None                    -- timeout -> test case fails
    except Exception:
        return None

```

6.3.5 `run_tests_verbose(code, inputs, outputs, timeout=2) → dict`

`run_tests_verbose(...)` -> dict

Called by: `ga._eval_code_for_problem()`; `main._holdout_one()`

Calls: `_compile()`, `_run_one()`, `_outputs_match()`

Returns a rich dict suitable for JSONL logging:

```

{
    "score":          float,          -- passed / total
    "reason":         str,             -- "empty_code" | "compile_fail" | "no_tests"
                                     -- | "perfect" | "partial_pass"
                                     -- | "all_fail" | "all_fail_early_stop"
    "compile":        {"success": bool, "error": str|None, "time_s": float},
    "test_cases":     [{"index": int, "passed": bool, "actual": str|None,
                        "expected": str, "time_s": float, "timeout": bool}, ...],
    "tests_total":    int,             -- total available test cases
    "tests_run":      int,             -- how many actually executed (early-stop)
    "early_stopped":  bool,
    "total_test_time_s": float,
}

```

Early stopping

After `EARLY_STOP_FAILURES=3` consecutive failures, the test loop exits. The score denominator is still `len(inputs)` (total), so skipped tests count as failures. This saves time on clearly broken solutions without giving them a free pass.

`run_tests()` is a thin backward-compatible wrapper:

```
def run_tests(code, inputs, outputs, timeout=2) -> float:
    return run_tests_verbose(code, inputs, outputs, timeout)["score"]
```


Chapter 7

Cost Tracking: `cost_tracker.py`

7.1 Purpose

Logs token counts and estimated costs for every Vertex AI API call. The tracker *streams* each call record to a JSONL file immediately (no in-memory list), keeping memory usage flat regardless of run length. Running totals are maintained as simple scalars. A summary JSON is written to `Costs/` at the end of each run and also live-updated after every call for real-time monitoring.

Real charges

The project calls Google Cloud Vertex AI, which bills real charges to your GCP account. The PRICING table lists the actual Gemini rates (USD per million tokens) as of April 2026. The \$300 GCP free credit applies to these charges. Thinking tokens (internal chain-of-thought) are billed at the same output token rate as regular output tokens and are aggregated into `output_tokens` before cost calculation.

7.2 The PRICING Table

```
PRICING = {
    # model: (input_$/MTok, output_$/MTok)
    # Gemini on Vertex AI (active models)
    "gemini-2.5-flash":      (0.15, 0.60),
    "gemini-2.5-pro":       (1.25, 10.00),
    "gemini-2.0-flash":     (0.10, 0.40),
    "gemini-2.0-flash-001": (0.10, 0.40),
    # Claude on Vertex AI (kept for reference)
    "claude-haiku-4-5-20251001": (1.00, 5.00),
    "claude-sonnet-4-6":      (3.00, 15.00),
}
```

In practice only `gemini-2.5-flash` and `gemini-2.5-pro` are used.

7.3 CallRecord — One Call's Data

```
@dataclass
class CallRecord:
    phase: str          # "fitness", "crossover", "mutate_inject", etc.
    model: str
    input_tokens: int
    output_tokens: int  # already includes thinking tokens (summed by llm.py)
    cost_usd: float | None = None  # set directly from API response

    @property
    def total_cost(self) -> float:
        if self.cost_usd is not None:
            return self.cost_usd  # prefer the value passed in from llm.py
        rate_in, rate_out = PRICING.get(self.model, (0, 0))
```

```

return (self.input_tokens * rate_in
        + self.output_tokens * rate_out) / 1_000_000

```

7.4 CostTracker — The Streaming Singleton

```
class CostTracker
```

Module-level singleton: `tracker = CostTracker()`

Imported by `llm.py` as: `from cost_tracker import tracker`

```

CostTracker.set_log_path(path)
    Called by: main.main() before GA starts
    Action: sets path for streaming JSONL writes (calls.jsonl)

CostTracker.set_live_path(path)
    Called by: main.main()
    Action: path for live JSON updates inside the run directory

CostTracker.set_costs_live_path(path)
    Called by: main.main()
    Action: second live path (in Costs/ folder) -- updated after every call

CostTracker.restore_from_jsonl(path) -> int
    Called by: main.main() on resume
    Action: reloads running totals from an existing calls.jsonl
    Returns: number of lines loaded (for logging)

CostTracker.record(phase, model, input_tokens, output_tokens, cost_usd)
    Called by: llm._call_realtime_api() after every successful API response
    Thread-safe (internal _lock)
    Action:
        - updates running totals (_total_calls, _total_cost_usd, etc.)
        - writes one JSON line immediately to calls.jsonl:
          {"t":"HH:MM:SS", "phase":..., "model":..., "in":..., "out":...,
           "cost":..., "total_calls":..., "running_cost":...}
        - overwrites live summary files (costs_live.json, Costs/cost_<run>.json)

CostTracker.flush(extra=None) -> Path
    Called by: main.main() at the very end
    Action:
        - assembles summary from running totals (_phase_totals, _model_totals)
        - writes Code/Costs/cost_TIMESTAMP.json
        - returns Path of the written file

```

Streaming vs. in-memory

Each `CallRecord` is written to disk immediately and the object is discarded. Running totals (`_total_calls`, `_total_input_tokens`, `_phase_totals`, `_model_totals`) are simple scalars/dicts updated in-place. A run with 4,425 API calls uses negligible memory for cost tracking — there is no growing list of records.

Chapter 8

LLM Calls: `llm.py`

8.1 Purpose

The sole interface to Gemini models via Google Cloud Vertex AI. All public functions route through the private `_call_realtime_api()` wrapper, which:

1. Obtains a shared `google.genai.Client` (lazy singleton, thread-safe).
2. Calls `client.models.generate_content()` with the appropriate config.
3. Tracks thinking tokens separately and combines them into the output token count.
4. Retries on rate limits, server errors, and timeouts with exponential backoff.
5. Records token counts and cost to the `CostTracker` singleton.

8.2 File at a Glance

Item	Type	Description
<code>LLMCallError</code>	exception	Raised when all retries fail for <code>generate_code()</code>
<code>set_error_log_path()</code>	function	Set path for <code>llm_errors.log</code>
<code>_get_api_client()</code>	private	Thread-safe lazy singleton: returns Vertex AI client
<code>_call_realtime_api()</code>	private	Actual API call with retry & cost tracking
<code>_call()</code>	private	Router to <code>_call_realtime_api()</code>
<code>generate_code()</code>	function	Ask <code>TARGET_MODEL</code> to solve a Codeforces problem
<code>run_fitness_batch()</code>	function	Stub: batch API not used (raises <code>NotImplementedError</code>)
<code>crossover()</code>	function	Ask <code>OPTIMIZER_MODEL</code> to merge two prompts
<code>mutate_inject()</code>	function	Ask <code>OPTIMIZER</code> to add one new strategy to a prompt
<code>mutate_delete()</code>	function	Ask <code>OPTIMIZER</code> to remove the weakest sentence
<code>mutate_rephrase()</code>	function	Ask <code>OPTIMIZER</code> to rephrase without changing meaning

Table 8.1: `llm.py`: all items.

8.3 Output Directives

Two constant strings are appended to every prompt at call time to enforce output format:

```
_CODE_GEN_DIRECTIVE = (
    "\n\nABSOLUTE OUTPUT REQUIREMENT -- NO EXCEPTIONS:\n"
    "Your entire response must consist of ONLY the raw C++ source code -- "
    "one complete file, nothing else. Use C++23.\n"
    "FORBIDDEN:\n"
    " * Backtick code fences of any form (no ‘‘‘, no ‘, no ‘cpp’ tags)\n"
    " * Markdown formatting of any kind\n"
```

```

" * Any text before or after the code\n"
"The FIRST character of your response must be the first character of the "
"C++ source code. The LAST character must be the last character of the code.
"
)

_OPERATOR_DIRECTIVE = (
    "\n\nABSOLUTE OUTPUT REQUIREMENT -- NO EXCEPTIONS:\n"
    "Your entire response must consist of ONLY the plain text of the instruction
    "
    "prompt -- nothing else.\n"
    "FORBIDDEN: backtick fences, markdown, labels, meta-commentary."
)

```

These are appended at call time, keeping the chromosome text clean — evolved prompts never contain operational directives.

8.4 _get_api_client()

`_get_api_client()` -> `google.genai.Client`

Called by: `_call_realtime_api()` (once per process, then cached)

Thread-safe: double-checked locking via `_api_client_lock`

```

def _get_api_client():
    global _api_client
    if _api_client is not None:
        return _api_client          # fast path: already initialised
    with _api_client_lock:
        if _api_client is not None:
            return _api_client      # double-check: another thread may have beaten
                                   us

        sa_info = json.loads(config.SERVICE_ACCOUNT_PATH.read_text())
        project_id = sa_info["project_id"]
        credentials = service_account.Credentials.from_service_account_info(
            sa_info,
            scopes=["https://www.googleapis.com/auth/cloud-platform"],
        )
        _api_client = genai.Client(
            vertexai=True,
            project=project_id,
            location=config.GCP_REGION,          # "us-central1"
            credentials=credentials,
            http_options=HttpOptions(timeout=120_000), # 120s per request
        )
    return _api_client

```

The 120-second HTTP timeout is set at the client level. If any individual API request takes longer, the SDK raises a timeout exception, which the retry loop catches and handles with exponential backoff.

8.5 `_call_realtime_api()` — Core API Wrapper

`_call_realtime_api(phase, model, system, user, max_tokens, retries=10) -> Optional[str]`

Returns: response text on success; "" if all retries return empty; None if all retries fail

```
_call_realtime_api(phase, model, system, user, max_tokens, retries=10):

    client = _get_api_client()

    for attempt in range(10):
        t0 = time.monotonic()
        try:
            is_fitness = (phase == "fitness")
            response = client.models.generate_content(
                model=model,
                contents=user,
                config=GenerateContentConfig(
                    system_instruction=system,
                    max_output_tokens=max_tokens,
                    temperature=0.0 if is_fitness else 0.7,
                    thinking_config=ThinkingConfig(
                        thinking_budget=512 if is_fitness else 2048,
                    ),
                ),
            )

            text = response.text or ""

            # -- token accounting --
            usage = response.usage_metadata
            input_toks = usage.prompt_token_count or 0
            output_toks = usage.candidates_token_count or 0
            think_toks = usage.thoughts_token_count or 0
            output_toks += think_toks # thinking billed at output rate; merge

            cost = _compute_gemini_cost(model, input_toks, output_toks)
            tracker.record(phase=phase, model=model,
                           input_tokens=input_toks, output_tokens=output_toks,
                           cost_usd=cost)

            if not text.strip():
                -- empty response
                wait = 5 * (2**attempt) + random.uniform(0, 3)
                time.sleep(wait); continue

            return text -- SUCCESS

        except Exception as exc:
            -- classify: is_rate (429/RESOURCE_EXHAUSTED), is_server (500/503),
            -- is_timeout ("timeout" / "timed out")
            -- all retryable: wait = 5 * 2^attempt + jitter; time.sleep(wait)
            -- non-retryable (auth, 404, etc.): still retry with same backoff

    return None -- all retries exhausted
```

Thinking token accounting

Gemini 2.5 models support extended thinking. The model's internal chain-of-thought appears in `usage_metadata.thoughts_token_count` but is *not* included in `candidates_token_count`. The code explicitly adds them: `output_toks += think_toks` before computing cost. This means the cost figure includes both the visible response and the hidden reasoning.

8.6 `generate_code(system_prompt, question) → str`

`generate_code(system_prompt, question) → str`

Called by: `ga._eval_one_problem()`; `main._holdout_one()`

Raises: `LLMCallError` if all retries fail (returns `None` from `_call`)

```
def generate_code(system_prompt, question) -> str:
    user_msg = _build_fitness_user_msg(question)
    # _build_fitness_user_msg appends:
    #   "Write a complete, compilable C++ program that reads from stdin and
    #   writes the answer to stdout. Include main(). Compile target: g++ -O2
    #   -std=c++23. DO NOT write any explanation ... Your response must begin
    #   with #include or int main -- nothing before it."
    result = _call(
        phase="fitness",
        model=config.TARGET_MODEL,                # "gemini-2.5-flash"
        system=system_prompt + _CODE_GEN_DIRECTIVE,
        user=user_msg,
        api_max_tokens=config.API_FALLBACK_MAX_TOKENS_CODE, # 8192
    )
    if result is None:
        raise LLMCallError(
            f"generate_code: all retries failed (model={config.TARGET_MODEL})"
        )
    return result
```

8.7 `crossover()` and the Three Mutation Operators

```
crossover(prompt_a, prompt_b) -> str    [phase="crossover", OPTIMIZER_MODEL]
System: "You merge coding-instruction prompts. Output ONLY the merged prompt."
User:   "Combine these two into one coherent prompt.
        Preserve the strongest strategies from each parent.
        Do not simply concatenate.
        -- Parent A -- {parent_a}
        -- Parent B -- {parent_b}"

mutate_inject(prompt, failure_examples=None) -> str
                                                [phase="mutate_inject",
                                                OPTIMIZER_MODEL]

If failure_examples given:
    Appends: "Recent failure examples (address at least one):
              - Problem X (rating R): reason=compile_fail, ..."
User: "Add one new, specific coding strategy ...
      Examples: 'validate your output', 'consider time complexity', ..."
```

```
mutate_delete(prompt) -> str                                [phase="mutate_delete",
    OPTIMIZER_MODEL]
Edge case FIRST:
    sentences = re.split(r"(?<=[.!?])\s+", prompt)
    if len(sentences) <= 1: return prompt    -- skip CLI call for single
        sentence
    User: "Remove one sentence -- the one that contributes the least."

mutate_rephrase(prompt) -> str                                [phase="mutate_rephrase",
    OPTIMIZER_MODEL]
    User: "Rephrase using different words, keep same meaning and strategies."
```

Failure examples in mutate_inject

When `ga.breed()` calls `mutate_inject()`, it passes up to 5 failure examples collected from the two parents during their fitness evaluation. These describe actual failures: compile errors, wrong answers, timeouts. The Optimizer model sees this context and is instructed to add a strategy targeting those specific failures. This creates a feedback loop: the GA learns from where the current best prompts fail.

Chapter 9

Genetic Algorithm: `ga.py`

9.1 Purpose

Implements the full evolutionary loop: seed evaluation, tournament selection, crossover/mutation breeding, parallel fitness evaluation, convergence detection, and crash-recovery checkpoints. Also provides RNG state serialization for deterministic resume.

9.2 File at a Glance

Item	Type	Description
Individual	dataclass	One chromosome (prompt + metadata)
<code>rng_state_to_dict()</code>	function	Serialize MT19937 RNG state to JSON-safe dict
<code>rng_state_from_dict()</code>	function	Deserialize RNG state dict back to tuple
<code>_eval_code_for_problem()</code>	private	Extract + compile + test one response; write JSONL
<code>_eval_one_problem()</code>	private	Call Gemini for one problem; delegate to <code>_eval_code_for_problem</code>
<code>compute_fitness()</code>	function	Evaluate one individual on all 65 problems (parallel)
<code>_assign_failure_examples()</code>	private	Keep diverse subset of failure examples
<code>tournament_select()</code>	function	Pick best of TOURNAMENT_SIZE random draws
<code>roulette_select()</code>	function	Fitness-proportionate (roulette-wheel) selection
<code>breed()</code>	function	Produce one offspring via crossover + mutation
<code>run()</code>	function	Main GA loop (init, evolve, converge, return best); accepts selection scheme

Table 9.1: `ga.py`: all items.

9.3 Individual — The Chromosome Dataclass

```
@dataclass
class Individual:
    prompt: str # the system prompt (the chromosome)
    fitness: float = -1.0 # -1.0 = not yet evaluated
    uid: str = field(default_factory=lambda: uuid.uuid4().hex
                     [:8])
    parent_a: str = "" # uid of first parent (empty for seeds)
    parent_b: str = "" # uid of second parent (empty for seeds)
    operators: str = "" # e.g. "crossover+inject" or "seed"
    intermediate_steps: dict = field(default_factory=dict)
    # Records the prompt after each operator:
    # {"after_crossover": "...", "after_inject": "...", "after_delete": "..."}
    failure_examples: list = field(default_factory=list)
```



```
# Up to 5 dicts describing problems where this individual scored < 1.0:
# [{"problem_id": "train/0002", "rating": 800, "reason": "compile_fail",
#   "compile_error": "...", "expected": "...", "actual": "..."}, ...]
```

9.4 _eval_code_for_problem() and _eval_one_problem()

`_eval_one_problem(gen, individual, prob, idx, total) -> (float, dict|None)`

Called by: `compute_fitness()` via `ThreadPoolExecutor`

Calls: `llm.generate_code()`, then `_eval_code_for_problem()`

```
_eval_one_problem(gen, individual, prob, idx, total):
    time.sleep(random.uniform(0, 1))    -- stagger concurrent API requests

    t_llm = time.monotonic()
    try:
        response = llm.generate_code(individual.prompt, prob["question"])
    except llm.LLMCallError as exc:
        llm_time = monotonic() - t_llm
        return _eval_code_for_problem(gen, individual, prob, idx, total,
                                      None, llm_time)    -- None = infra failure

    llm_time = round(monotonic() - t_llm, 3)
    return _eval_code_for_problem(gen, individual, prob, idx, total,
                                  response, llm_time)

_eval_code_for_problem(gen, individual, prob, idx, total, response_text,
                      llm_time_s):
    if response_text is None:            -- infra failure path
        _log_evaluation({... , score:-1.0, reason:"infra_failure"})
        return -1.0, None

    code = evaluator.extract_code(response_text); del response_text

    if not code.strip():
        test_result = {score:0.0, reason:"empty_response", ...}
    else:
        test_result = evaluator.run_tests_verbose(code, prob["inputs"],
                                                  prob["outputs"], EXEC_TIMEOUT)

    _log_evaluation({ts, gen, uid, operators, system_prompt, problem_id,
                    rating, llm_time, extracted_code, compile, test_cases,
                    tests_total, tests_run, early_stopped, score, reason})
    del entry, code

    -- build failure_example if score < 1.0 (not infra_failure):
    failure_example = {problem_id, rating, reason,
                      expected[:200], actual[:200], compile_error[:200]}

    return score, failure_example    -- (float, dict|None)
```

9.5 compute_fitness(gen, individual, problems) → float

compute_fitness(...) → float in [-1.0, 1.0]

Called by: run() — 5 times at gen 0; 5 times per generation (gens 1–12)

Parallelism: ThreadPoolExecutor(max_workers=PARALLEL_EVALS=5)

```
def compute_fitness(gen, individual, problems):
    scores = [0.0] * len(problems)
    collected_failures = []
    _failures_lock = threading.Lock()

    with ThreadPoolExecutor(max_workers=config.PARALLEL_EVALS) as executor:
        futures = {
            executor.submit(_eval_one_problem, gen, individual, prob, i,
                            len(problems)): i
            for i, prob in enumerate(problems)
        }
        completed = 0
        for future in as_completed(futures):
            idx = futures[future]
            try:
                score, failure_example = future.result()
                scores[idx] = score
                if failure_example is not None:
                    with _failures_lock:
                        collected_failures.append(failure_example)
            except Exception as exc:
                scores[idx] = -1.0
            completed += 1
            if completed % 10 == 0 or completed == len(problems):
                _log(f"ind={individual.uid} progress {completed}/{len(problems)}")

    _assign_failure_examples(individual, collected_failures)
    fitness = sum(scores) / len(scores)
    return fitness
```

9.6 _assign_failure_examples(individual, collected_failures)

Keeps ≤5 failure examples, prioritising diversity of failure reason:

```
_reason_priority = {
    "compile_fail": 0,      # highest priority -- most informative
    "all_fail": 1,
    "all_fail_early_stop": 1,
    "partial_pass": 2,
}

# Sort by priority; then include an example of each reason type.
# At most 5 examples total; at most 3 per reason (to ensure diversity).
```

These failure examples are later passed to `llm.mutate_inject()` to focus new strategy additions on the actual failure modes.

9.7 RNG State Serialization

```
def rng_state_to_dict(state: tuple) -> dict:
    version, internalstate, gauss_next = state
    return {
        "version": version,
        "internalstate": list(internalstate), # tuple -> list for JSON
        "gauss_next": gauss_next,
    }

def rng_state_from_dict(d: dict) -> tuple:
    return (d["version"], tuple(d["internalstate"]), d["gauss_next"])
```

The MT19937 RNG state is saved in `generation_NN.json` checkpoints and restored on resume, making tournament selection decisions deterministic across restarts.

9.8 tournament_select, roulette_select, and breed

```
tournament_select(population, rng):
    contestants = rng.sample(population, TOURNAMENT_SIZE=3) -- no replacement
    return max(contestants, key=lambda ind: ind.fitness)

roulette_select(population, rng):
    fitnesses = [max(ind.fitness, 0.0) for ind in population]
    total = sum(fitnesses)
    if total == 0:
        return rng.choice(population) -- fallback: uniform when all scores 0
    r = rng.uniform(0, total)
    cumulative = 0.0
    for ind, f in zip(population, fitnesses):
        cumulative += f
        if cumulative >= r:
            return ind
    return population[-1] -- floating-point safety fallback

breed(parent_a, parent_b, rng, allow_mutation=True):

    [Collect combined failure examples from both parents (up to 5 unique)]

    [CROSSOVER]
    if rng.random() < 0.80:
        result = llm.crossover(parent_a.prompt, parent_b.prompt)
        prompt = result if result.strip() else rng.choice([a.prompt, b.prompt])
        ops = ["crossover"]
    else:
        prompt = rng.choice([parent_a.prompt, parent_b.prompt])
        ops = ["copy"]

    steps["after_crossover"] = prompt

    [MUTATION -- only when allow_mutation=True]
    if allow_mutation:
        if rng.random() < 0.25:
            result = llm.mutate_inject(prompt, failure_examples=combined)
            prompt = result if result.strip() else prompt
            ops.append("inject")
            steps["after_inject"] = prompt

        if rng.random() < 0.15:
            result = llm.mutate_delete(prompt)
            prompt = result if result.strip() else prompt
```

```

        ops.append("delete")
        steps["after_delete"] = prompt

    if rng.random() < 0.10:
        result = llm.mutate_rephrase(prompt)
        prompt = result if result.strip() else prompt
        ops.append("rephrase")
        steps["after_rephrase"] = prompt

    return Individual(prompt, parent_a=a.uid, parent_b=b.uid,
                      operators="+".join(ops), intermediate_steps=steps)

```

9.9 run() — The Main GA Loop

```
run(seed_prompts, problems, run_dir, on_generation, resume_*,
    selection_scheme, elitism_count) -> (Individual, list[dict])
```

Called by: main.main()

New parameters:

selection_scheme — "tournament" (default), "roulette", or "elitism"

elitism_count — overrides config.ELITISM_COUNT when scheme is "elitism" (pass 1)

```

run(seed_prompts, problems, run_dir, on_generation,
    resume_population=None, resume_start_gen=0,
    resume_history=None, resume_rng_state=None,
    selection_scheme="tournament", elitism_count=None):

    _progress_log = run_dir / "progress.log"
    _eval_log_dir = run_dir

    -- resolve selection function
    _scheme = selection_scheme.lower()
    if _scheme == "roulette":
        _select = roulette_select
    else:
        _select = tournament_select          -- "tournament" or "elitism"

    _elitism = elitism_count if elitism_count is not None else config.
        ELITISM_COUNT

    rng = random.Random(RANDOM_SEED=42)
    history = list(resume_history) if resume_history else []

    [INIT OR RESUME]
    if resume_population is not None:
        population = resume_population
        if resume_rng_state: rng.setstate(rng_state_from_dict(resume_rng_state))
        loop_start = resume_start_gen + 1
    else:
        population = [Individual(prompt=p, operators="seed") for p in seed_prompts
                      [:5]]

    -- load per-seed checkpoints from previous crashed run
    existing_seeds: dict[int, dict] = {ckpt.seed_idx: ckpt for ckpt in
        seed_NN_*.json}

    for seed_idx, ind in enumerate(population):
        if seed_idx in existing_seeds:

```

```

        restore ind.uid, fitness, failure_examples from checkpoint
        continue
    ind.fitness = compute_fitness(0, ind, problems)    -- 65 Gemini Flash
    calls
    save seed checkpoint: run_dir/seed_NN_<uid>.json    -- crash recovery

    loop_start = 1

stagnation_count = 0

[GENS loop_start .. 12]
for gen in range(loop_start, GENERATIONS + 1):

    population.sort(by fitness, descending)
    fits = [ind.fitness for ind in population]
    stats = {generation:gen-1, best, mean, worst, std, cv, best_uid,
             best_prompt}
    if stats.generation > history[-1].generation:
        history.append(stats)

    on_generation(gen-1, population, stats, rng.getstate())
        -> save_checkpoint(run_dir, gen-1, pop, stats, rng_state)

    [CONVERGENCE CHECK]
    cv = pop_std / pop_mean (if pop_mean > CONVERGENCE_MIN_MEAN=0.01)
    if cv < CONVERGENCE_CV_THRESHOLD=0.01:
        stagnation_count += 1
        if stagnation_count >= CONVERGENCE_PATIENCE=3:
            break    -- EARLY STOP
    else:
        stagnation_count = 0

    [ELITISM -- _elitism frozen elites (0 in default scheme)]
    new_pop = population[:_elitism]

    [BREED remaining slots]
    offspring_idx = 0
    while len(new_pop) < POPULATION_SIZE:
        p1 = _select(population, rng)    -- tournament or roulette
        p2 = _select(population, rng)
        allow_mut = (offspring_idx >= CROSSOVER_ONLY_SLOTS=2)
        child = breed(p1, p2, rng, allow_mutation=allow_mut)
        new_pop.append(child)
        offspring_idx += 1

    [EVALUATE]
    for off_idx, ind in enumerate(new_pop):
        ind.fitness = compute_fitness(gen, ind, problems)

    population = new_pop
    population.sort(by fitness, descending)

    [FINAL STATS]
    population.sort()
    final_stats = {generation:last_completed_gen, best, mean, ...}
    history.append(final_stats)

return (population[0], history)

```

Generational replacement with no elites (default)

Unlike classic elitist GAs, every slot is a new offspring each generation. The best individual is not automatically preserved. This increases exploration at the cost of potentially losing a good solution temporarily. With `POPULATION_SIZE=5`, even one excellent offspring that emerges from crossover of the best parents is likely to dominate the next tournament selection.

Three selection schemes (`-scheme` argument)

The `selection_scheme` parameter (set via `-scheme` in `main.py`) controls how parents are chosen each generation:

- **tournament** (default): sample 3 at random, take the fittest. Strong selection pressure; fast; default for all experiments.
- **roulette**: fitness-proportionate selection. Individuals with higher fitness have a proportionally higher chance of being chosen. Falls back to uniform random when all fitnesses are zero.
- **elitism**: same tournament selection plus `elitism_count=1` frozen elite. The single fittest individual from the previous generation is carried over unchanged to guarantee no regression.

All three schemes share the same crossover and mutation pipeline.

Per-seed crash recovery

After each seed is evaluated at gen-0, its result is saved to a file named `seed_NN_<uid>.json`. If the process crashes during gen-0 (e.g. after seed 2 but before seed 3), the next run finds these checkpoints and skips the already-evaluated seeds. This prevents wasting API quota re-evaluating seeds that already succeeded.

Chapter 10

Orchestration: `main.py`

10.1 Purpose

The entry point. Handles pre-flight credential checks, dataset loading, run directory creation, GA execution, holdout evaluation, genealogy report generation, and cost report saving. Also implements crash-resume logic: a run can be resumed from any saved generation checkpoint, with deterministic RNG restoration.

10.2 File at a Glance

Item	Type	Description
<code>save_checkpoint()</code>	function	Serialize full population to <code>generation_NN.json</code>
<code>_find_latest_run()</code>	private	Find most recently modified <code>run_*</code> directory
<code>_load_resume_state()</code>	private	Load last checkpoint; reconstruct Individual objects
<code>_holdout_one()</code>	private	Evaluate one problem against the best prompt
<code>evaluate_on_holdout()</code>	function	Parallel holdout evaluation; returns mean score
<code>main()</code>	function	The 8-step orchestration pipeline

Table 10.1: `main.py`: all items.

10.3 Function-by-Function Analysis

10.3.1 `save_checkpoint(run_dir, gen, population, stats, rng_state=None)`

`save_checkpoint(...)` -> None

Called by: the `on_generation` callback in `main()`

Output: `results/run_*/generation_NN.json`

Saves the full population state (all 5 prompts, fitnesses, parent UIDs, operators, intermediate steps, failure examples) *and* the MT19937 RNG state:

```
data = {
    "generation": gen,
    "stats": stats,
    "rng_state": ga.rng_state_to_dict(rng_state),    # or None
    "population": [
        {"uid": ind.uid, "prompt": ind.prompt, "fitness": ind.fitness,
         "parent_a": ind.parent_a, "parent_b": ind.parent_b,
         "operators": ind.operators,
         "intermediate_steps": ind.intermediate_steps,
         "failure_examples": ind.failure_examples}
        for ind in population
    ]
}
```

```

    ],
}
gen_file.write_text(json.dumps(data, indent=2))

```

10.3.2 `_find_latest_run()` and `_load_resume_state()`

`_find_latest_run()` globs `config.RESULTS_DIR` for `run_*` directories and returns the one with the most recent `st_mtime`.

`_load_resume_state(run_dir)` reads all `generation_NN.json` files, extracts the last one's population, rebuilds `Individual` objects, and reconstructs the history list from all checkpoints. Also reads the saved RNG state for deterministic continuation.

```

_load_resume_state(run_dir) -> dict:
    checkpoints = sorted(run_dir.glob("generation_*.json"))
    if not checkpoints: raise FileNotFoundError(...)

    history = [data["stats"] for each checkpoint]
    history.sort(key=lambda s: s["generation"])

    last_ckpt = json.loads(checkpoints[-1].read_text())
    population = [
        ga.Individual(prompt=ind["prompt"], fitness=ind["fitness"],
                       uid=ind["uid"], parent_a=ind["parent_a"],
                       parent_b=ind["parent_b"], operators=ind["operators"],
                       intermediate_steps=ind["intermediate_steps"],
                       failure_examples=ind["failure_examples"])
        for ind in last_ckpt["population"]
    ]
    rng_state = last_ckpt.get("rng_state")
    evolution_ids = json.loads((run_dir/"evolution_ids.json").read_text())

    return {"population": population, "start_gen": last_gen,
            "history": history, "evolution_ids": evolution_ids,
            "rng_state": rng_state}

```

10.3.3 `evaluate_on_holdout(best_prompt, holdout, run_dir) → float`

```
evaluate_on_holdout(best_prompt, holdout, run_dir) -> float
```

Parallelism: `ThreadPoolExecutor(max_workers=PARALLEL_EVALS=5)`

Output: results appended to `holdout_evaluations.jsonl`

Evaluates the best prompt on 200 held-out problems in parallel. Each call to `_holdout_one()` sleeps a random 0–2 seconds to stagger API requests, then calls `llm.generate_code()` and `evaluator.run_tests_verbose()`. Results are appended to JSONL immediately (thread-safe lock). Returns the mean score.

10.3.4 `main()` — The 8-Step Pipeline

```

main():

    [1] Parse args
        --resume [RUN_DIR]    resume a crashed run (AUTO or explicit path)
        --scheme {tournament|roulette|elitism}
                               parent selection scheme (default: tournament)

```



```

[2] Pre-flight: Vertex AI credentials
    check SERVICE_ACCOUNT_PATH exists
    import google.genai, google.oauth2.service_account (verify installed)
    read service-account.json: log project_id, client_email

[3] Pre-flight: dataset dirs
    if not APPS_TRAIN.exists() or not APPS_TEST.exists(): sys.exit(1)

[4] Load dataset
    all_problems = dataset.load_problems()    -- 305 Codeforces problems

[5] Handle resume vs fresh run
    Fresh:
        evolution_pool, holdout, rest = dataset.split(all_problems, 65, 200,
            42)
        run_dir = RESULTS_DIR / "run_YYYYMMDD_HHMMSS_<scheme>"
            -- e.g. "run_20240501_143022_tournament"
        writes: config.json, evolution_ids.json

    Resume (--resume [path]):
        run_dir = specified path or _find_latest_run()
        if seed checkpoints exist but no generation checkpoints:
            resume_state = None -- ga.run() handles seed-level resume
        else:
            resume_state = _load_resume_state(run_dir)
        restore evolution_pool from evolution_ids.json

[6] Set up logging
    tracker.set_log_path(run_dir/"calls.jsonl")
    tracker.set_live_path(run_dir/"costs_live.json")
    tracker.set_costs_live_path(Costs/"cost_<run>.json")
    llm.set_error_log_path(run_dir/"llm_errors.log")
    if resume: tracker.restore_from_jsonl(run_dir/"calls.jsonl")

[7] Run GA
    best, history = ga.run(
        seed_prompts=SEED_PROMPTS[:POPULATION_SIZE],
        problems=evolution_pool,
        run_dir=run_dir,
        on_generation=lambda gen, pop, stats, rng_s:
            save_checkpoint(run_dir, gen, pop, stats, rng_s),
        resume_population=resume_state["population"] if resume_state else None,
        ,
        resume_start_gen= resume_state["start_gen"] if resume_state else 0,
        resume_history= resume_state["history"] if resume_state else None
        ,
        resume_rng_state= resume_state["rng_state"] if resume_state else None
        ,
        selection_scheme= args.scheme, -- "tournament"|"
            roulette|"elitism"
        elitism_count= 1 if args.scheme == "elitism" else None,
    )

[8] Save evolution results
    (run_dir/"history.json").write_text(history)
    (run_dir/"best_prompt.txt").write_text(best.prompt)

[9] Holdout evaluation
    holdout_fitness = evaluate_on_holdout(best.prompt, holdout, run_dir)
    (run_dir/"summary.json").write_text({elapsed, best_prompt,
        best_fitness_evolution, best_fitness_holdout, best_uid,
        total_generations, resumed})

```

```

[10] Genealogy report
    genealogy.generate_report(run_dir, best.uid)
    -> genealogy_report.txt + genealogy_tree.json

[11] Cost report
    cost_path = tracker.flush(extra={run_dir, backend, fitnesses, ...})
    -> Code/Costs/cost_TIMESTAMP.json

```

10.4 Output Files

File	Contents
run_*/config.json	Exact hyperparameters snapshot
run_*/evolution_ids.json	The 65 problem IDs in the evolution pool
run_*/calls.jsonl	Per-call token + cost log (streaming)
run_*/costs_live.json	Live cost summary (overwritten after every call)
run_*/llm_errors.log	API retry / error events
run_*/progress.log	Human-readable timestamped timeline
run_*/evaluations_genNN.jsonl	Per-problem full evaluation records
run_*/generation_NN.json	Full population state + RNG state per generation
run_*/seed_NN_<uid>.json	Per-seed checkpoint (gen 0 crash recovery)
run_*/history.json	Best/mean/worst/std/cv per generation
run_*/best_prompt.txt	The evolved prompt (main artefact)
run_*/summary.json	Final fitnesses (evolution + holdout), runtime
run_*/holdout_evaluations.jsonl	Per-problem holdout evaluation records
run_*/genealogy_report.txt	Human-readable ancestry chain of the best prompt
run_*/genealogy_tree.json	Machine-readable full ancestry tree
Costs/cost_*.json	Token counts and estimated cost breakdown

Table 10.2: All output files produced by one run.

Chapter 11

Genealogy: `genealogy.py`

11.1 Purpose

After the GA finishes, `genealogy.py` traces the ancestry of the best prompt backward through all checkpoint files, building a human-readable lineage report and a machine-readable full ancestry tree. This allows post-hoc analysis of how the best prompt evolved: which seed it descended from, which crossover created its key characteristics, and which mutations refined it.

11.2 File at a Glance

Item	Type	Description
<code>_load_all_individuals()</code>	private	Build a uid-keyed registry from all generation checkpoints
<code>_collect_lineage()</code>	private	Walk <code>parent_a</code> chain from best to seed
<code>_collect_full_tree()</code>	private	Recursively collect both parents at every fork
<code>generate_report()</code>	function	Entry point: produce <code>report.txt</code> and <code>tree.json</code>
<code>_write_text()</code>	private	Simple <code>path.write_text()</code> wrapper

Table 11.1: `genealogy.py`: all items.

11.3 Algorithm

```
generate_report(run_dir, best_uid):

[1] Load registry
    registry = {}
    for each generation_NN.json checkpoint:
        for each individual in checkpoint["population"]:
            if uid not in registry:
                registry[uid] = {uid, prompt, fitness, parent_a, parent_b,
                                operators, generation}

[2] Primary lineage (parent_a chain only)
    chain = []
    current = best_uid
    while current in registry and current not visited:
        chain.append(registry[current])
        current = registry[current]["parent_a"]

[3] Write genealogy_report.txt
    For each step in chain:
        print uid, generation, fitness, operators (role: BEST / SEED /
            ancestor N)
        print full prompt text
        print both parent uids + their fitness and generation
```

```
[4] Full ancestry tree (both parents at every fork)
def _collect_full_tree(uid, depth=0):
    if depth > 40 or not uid: return {}
    node = {uid, generation, fitness, operators, prompt_preview[:120]}
    if parent_a: node["parent_a"] = _collect_full_tree(parent_a, depth+1)
    if parent_b != parent_a: node["parent_b"] = _collect_full_tree(
        parent_b, depth+1)
    return node

tree = _collect_full_tree(best_uid)
write genealogy_tree.json (JSON, indented)
```

What the genealogy tells you

The `genealogy_report.txt` answers: “Where did the best prompt come from?” If the chain is: best → ancestor 1 → ancestor 2 → seed, you can read exactly which seed it descended from, which crossover blended two parent strategies, and which inject mutation added the critical strategy. The `operators` field at each step says what genetic operation produced that individual. The `genealogy_tree.json` shows the full binary tree — both parents at every fork — useful for quantifying how much of the seed population contributed to the final answer.

Glossary

API call budget

One full run makes $\approx 4,425$ Gemini Flash calls (fitness) and ≈ 66 Gemini Pro calls (crossover + mutation). See Chapter 2.

Chromosome A candidate solution in a GA. Here: a system prompt string, stored as an `Individual` dataclass.

Coefficient of Variation (CV)

$std/mean$ of fitnesses in the current population. Used by the convergence detector: $CV < 0.01$ for 3 consecutive generations triggers early stopping.

Convergence detection

Early stopping when the population becomes statistically homogeneous (CV below threshold for PATIENCE generations). Only checked when mean fitness $> CONVERGENCE_MIN_MEAN$.

CROSSOVER_ONLY_SLOTS

First $N = 2$ offspring per generation receive crossover only — no mutation. Preserves genetic material faithfully while still allowing recombination.

Crossover Genetic operator combining two parents into one offspring. Here: the Optimizer LLM (Gemini 2.5 Pro) semantically blends two parent prompts. Fires with probability 0.8.

Early stopping (test)

In `run_tests_verbose()`: halting the test loop after `EARLY_STOP_FAILURES=3` consecutive failures. Score denominator is still total tests.

Elitism Copying the top k individuals unchanged. `ELITISM_COUNT=0` in the current design — no frozen elites.

Evolution pool

The fixed 65 Codeforces problems used as the fitness benchmark, drawn by stratified sampling (seed 42).

Failure examples

Up to 5 problem-level failure records stored in each `Individual.failure_examples`. Passed to `mutate_inject()` to focus new strategy additions on actual failure modes.

Fitness Mean partial-credit score across 65 problems. Range: $[-1.0, 1.0]$ (infra failures score -1.0 ; otherwise $[0.0, 1.0]$).

Gemini Flash `gemini-2.5-flash` — the Target model. Used for code generation (fitness evaluation). Temperature 0.0, `thinking_budget` 512.

Gemini Pro `gemini-2.5-pro` — the Optimizer model. Used for crossover and mutation. Temperature 0.7, `thinking_budget` 2048.

Genealogy Ancestry report tracing the best evolved prompt back to its seed. Produced by `genealogy.py` after the GA completes.

Generational replacement

Every slot is filled by a new offspring each generation (no frozen elites). Creates strong selection pressure via tournament selection.

Held-out set 200 problems never seen during evolution. Used to test generalisation of the final evolved prompt.

Individual One GA population member: a prompt with fitness, uid, parent uids, operator history, intermediate steps, and failure examples.

Intermediate steps

`Individual.intermediate_steps`: dict recording the prompt text after each genetic operator (`after_crossover`, `after_inject`, etc.). Stored in checkpoints for post-hoc analysis.

Meta-prompt A prompt sent to the Optimizer LLM instructing it to modify another prompt (used in crossover and all three mutation operators).

Mutation Modifying a single chromosome. Three independent types: inject (25%), delete (15%), rephrase (10%). Only fires on offspring with index $\geq$ `CROSSOVER_ONLY_SLOTS`.

Partial credit Score = (tests passed) / (total tests). Provides a smooth gradient vs. binary pass/fail.

PARALLEL_EVALS

Number of problems evaluated simultaneously per individual (5, using a `ThreadPoolExecutor`).

Resume Running with `-resume` loads the last generation checkpoint, restores the population and RNG state, and continues from the next generation. Seed-level checkpoints (`seed_NN_*.json`) also allow gen-0 crash recovery.

RNG state The full MT19937 internal state serialized via `ga.rng_state_to_dict()`. Saved in each generation checkpoint; restored on resume to make tournament selection decisions deterministic.

Service account

A Google Cloud IAM service account JSON credential file (`service-account.json`). Used by `llm._get_api_client()` to authenticate with Vertex AI without interactive login.

Singleton The `tracker = CostTracker()` instance at module level; the `_api_client` Vertex AI client. Both shared across all threads.

Stratified split The evolution pool is built by drawing a fixed number of problems from each difficulty tier (800–1700) plus all harder problems (1800/1900), ensuring a consistent difficulty distribution.

Thinking tokens

Gemini 2.5 Flash/Pro internal chain-of-thought tokens. Billed at the output token rate. Tracked separately via `usage_metadata.thoughts_token_count` and added to output token count.

Tournament selection

Sample $k = 3$ individuals at random; return the fittest. Balances exploitation with exploration.

Two-model architecture

Flash (code generation, target) is optimised by the GA; Pro (reasoning, creative) performs the genetic operators.

Vertex AI

Google Cloud's managed AI platform. LLM calls use the `google-genai` Python SDK connected to the `us-central1` endpoint with service account credentials.